

Guia de instalação e referência rápida de comandos

Minicurso LaPSEE — Redes elétricas como grafos,
preços nodais e econometria espacial

LaPSEE · UNESP Ilha Solteira

Sobre este guia

Este documento acompanha os notebooks e laboratórios do minicurso. Ele tem três objetivos:

1. garantir que o seu ambiente Python esteja pronto **antes** da primeira aula (Seção 1);
2. apresentar o **pandapower** — a biblioteca de análise de sistemas de potência mais usada no ecossistema Python e, para a maioria de vocês, um primeiro contato (Seção 2);
3. apresentar o **ecossistema PySAL** (**libpysal**, **esda**, **spreg**) para econometria espacial — as ferramentas mais novas do curso, que merecem atenção redobrada (Seção 3).

As Seções 4–6 trazem uma página de **networkx**, o mapa “comando → onde aparece no curso” e a solução dos problemas mais comuns.

Versões testadas. O material foi validado com as versões abaixo. Versões mais novas em geral funcionam; versões *muito* antigas de **pandapower** (1.x) não.

Pacote	Versão testada	Pacote	Versão testada
Python	3.10 – 3.12	libpysal	4.14
numpy	2.x	esda	2.9
pandas	2.x	spreg	1.9
matplotlib	3.8+	scipy	1.13+
networkx	3.x	jupyter	qualquer recente
pandapower	3.x		

1 Instalação

Escolha **uma** das três opções. Se nunca configurou Python, a Opção C (Colab) é a de menor atrito; a Opção A é a recomendada para quem vai continuar usando as ferramentas depois do curso.

1.1 Opção A — pip com ambiente virtual (recomendada)

Um *ambiente virtual* isola os pacotes do curso do resto do seu sistema.

Windows (PowerShell):

```
python -m venv lapsee
lapsee\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install numpy pandas matplotlib networkx scipy jupyter
pip install pandapower libpysal esda spreg
```

Linux / macOS (terminal):

```
python3 -m venv lapsee
source lapsee/bin/activate
python -m pip install --upgrade pip
pip install numpy pandas matplotlib networkx scipy jupyter
pip install pandapower libpysal esda spreg
```

Para abrir os notebooks, com o ambiente ativado:

```
jupyter lab          # ou: jupyter notebook
```

Armadilha! Em distribuições Linux recentes (Ubuntu 24+, Debian 12+), instalar com **pip** fora de um **venv** exige a flag **pip install --break-system-packages** Prefira sempre o **venv**, que dispensa a flag.

1.2 Opção B — Anaconda / Miniconda

```
conda create -n lapsee python=3.12
conda activate lapsee
conda install numpy pandas matplotlib networkx scipy jupyter
pip install pandapower libpysal esda spreg
```

Dica. **pandapower** e **spreg** são instalados via **pip** mesmo dentro do **conda** — é o caminho oficial e evita conflitos de canal.

1.3 Opção C — Google Colab (sem instalar nada)

Abra <https://colab.research.google.com>, crie um notebook e rode na primeira célula:

```
!pip install -q pandapower libpysal esda spreg
```

Tudo o mais (numpy, pandas, networkx...) o Colab já traz. A única desvantagem: a instalação se repete a cada nova sessão.

1.4 Teste de fumaça: rode isto antes da aula

Copie o bloco abaixo em um notebook (ou em **teste.py**) e execute. Ele valida os imports, resolve um despacho ótimo de verdade e calcula um I de Moran. Se a última linha imprimir **TUDO OK**, você está pronto.

```
import numpy as np, pandas as pd, networkx as nx
import pandapower as pp, pandapower.networks as pn
from libpysal.weights import lat2W
from esda.moran import Moran
import spreg

print("pandapower", pp.__version__)
```

```
# 1) um OPF DC de verdade no IEEE 30
net = pn.case_ieee30()
pp.rundcopp(net)
assert net.res_bus["lam_p"].notna().all() # LMPs existem

# 2) um Moran de brinquedo numa grade 5x5
w = lat2W(5, 5); w.transform = "r"
rng = np.random.default_rng(0)
I = Moran(rng.normal(size=25), w, permutations=99).I

# 3) uma regressao espacial minima
y = rng.normal(size=(25, 1)); X = rng.normal(size=(25, 2))
m = spreg.ML_Lag(y, X, w=w, method="full")
print("TUDO OK - rho de teste:", round(float(m.rho), 3))
```

Armadilha! Na primeira execução podem aparecer avisos do numba ou do pandapower (sobre *lightsim2grid* ou conversores). São apenas avisos — se o script terminou com TUDO OK, ignore-os.

2 pandapower: primeiro contato

O **pandapower** representa uma rede elétrica como um objeto **net** que é, essencialmente, uma coleção de **DataFrames** do **pandas**: uma tabela por tipo de elemento. Tudo o que você sabe de **pandas** (indexar, filtrar, **groupby**) vale dentro dele — essa é a razão da sua popularidade.

2.1 O modelo mental: tabelas de entrada e de resultado

Tabela	Conteúdo	Colunas que usamos no curso
<code>net.bus</code>	barras	<code>vn_kv</code> , <code>geo</code> (coordenadas)
<code>net.line</code>	linhas	<code>from_bus</code> , <code>to_bus</code> , <code>x_ohm_per_km</code> , <code>length_km</code> , <code>max_i_ka</code>
<code>net.trafo</code>	transformadores	<code>hv_bus</code> , <code>lv_bus</code> , <code>vk_percent</code> , <code>sn_mva</code> , <code>tap_pos</code>
<code>net.load</code>	cargas	<code>bus</code> , <code>p_mw</code>
<code>net.gen</code>	geradores	<code>bus</code> , <code>p_mw</code> , <code>min/max_p_mw</code> , <code>controllable</code>
<code>net.ext_grid</code>	rede externa (<i>slack</i>)	<code>bus</code> , <code>min/max_p_mw</code>
<code>net.poly_cost</code>	custos de oferta	<code>element</code> , <code>et</code> , <code>cp1_eur_per_mw</code>
<code>net.res_bus</code>	resultados por barra	<code>va_degree</code> (ângulo), <code>lam_p</code> (= LMP!)
<code>net.res_line</code>	resultados por linha	<code>p_from_mw</code> , <code>loading_percent</code>
<code>net.res_cost</code>	escalar	custo total do despacho [$\$/h$]

As tabelas `res_*` só existem depois de rodar um solver (próxima seção).

2.2 Os comandos que usamos no curso

1. **Carregar um caso de teste.** O módulo `pandapower.networks` traz dezenas de redes clássicas prontas:

```
import pandapower as pp
import pandapower.networks as pn

net = pn.case_ieee30() # IEEE 30 (Sessoes 1 e 2)
net = pn.case39()      # New England 39 (Labs 1, 2 e 4)
net = pn.case57()      # IEEE 57 (Labs 1 e 2)
net = pn.case118()     # IEEE 118 (Sessao 2, siting)
```

2. Fluxo de potência DC — resolve $P = B\theta$ e preenche ângulos e fluxos:

```
pp.rundcpp(net)
net.res_bus["va_degree"]      # angulos [graus]
net.res_line["loading_percent"] # % de carregamento
```

3. Despacho ótimo DC (OPF) — minimiza o custo respeitando os limites. É daqui que saem os preços nodais:

```
pp.rundcpp(net)
lmp = net.res_bus["lam_p"]      # LMP [$ /MWh]: a variavel dual!
custo = float(net.res_cost)     # custo total [$ /h]
n_sat = int((net.res_line["loading_percent"] > 99.9).sum())
```

4. Apertar os limites térmicos. Os casos IEEE vêm com linhas quase infinitas (max_i_ka enorme) — sem limites não há congestionamento nem separação de preços. A conversão de uma capacidade S [MVA] para corrente é $I_{\max} = S/(\sqrt{3}V)$:

```
import numpy as np
for i, ln in net.line.iterrows():
    vn = float(net.bus.loc[int(ln.from_bus), "vn_kv"]) # kV
    S_mva = 50.0 # limite escolhido
    net.line.loc[i, "max_i_ka"] = S_mva / (np.sqrt(3) * vn)
net.line["max_loading_percent"] = 100.0
net.trafo["max_loading_percent"] = 100.0
```

5. Definir os custos de oferta dos geradores (necessário para o OPF):

```
net.poly_cost.drop(net.poly_cost.index, inplace=True) # limpa os custos do caso
pp.create_poly_cost(net, 0, "ext_grid", cp1_eur_per_mw=18.0)
for gi, g in net.gen.iterrows():
    pp.create_poly_cost(net, gi, "gen", cp1_eur_per_mw=45.0)
```

6. Criar elementos novos (usado nos what-ifs dos Labs 2 e 4):

```
# uma usina nova de 50 MW a $5/MWh na barra 16
gi = pp.create_gen(net, bus=16, p_mw=0.0, min_p_mw=0.0,
                  max_p_mw=50.0, controllable=True)
pp.create_poly_cost(net, gi, "gen", cp1_eur_per_mw=5.0)

# uma linha nova entre as barras 5 e 10
pp.create_line_from_parameters(net, 5, 10, length_km=1.0,
                              r_ohm_per_km=0.5, x_ohm_per_km=30.0, c_nf_per_km=0.0,
                              max_i_ka=600 / (np.sqrt(3) * 345))
```

7. Cenários horários — escalar todas as cargas e repetir o OPF:

```
cargas_base = net.load["p_mw"].values.copy()
for mult in [0.7, 0.85, 1.0]:
    net.load["p_mw"] = cargas_base * mult
    try:
        pp.rundcpp(net)
        print(mult, float(net.res_cost))
    except Exception:
        print(mult, "hora infactivel (corte de carga)")
net.load["p_mw"] = cargas_base # SEMPRE restaurar!
```

2.3 As cinco armadilhas clássicas

Armadilha! `lam_p` só existe depois do `rundcopp`. O fluxo DC simples (`rundcpp`) não produz preços — não há otimização, logo não há dual.

Armadilha! Custos pré-existentes. Vários casos já trazem uma tabela `poly_cost` (às vezes degenerada). Antes de definir as suas ofertas, limpe-a com `net.poly_cost.drop(net.poly_cost.index, inplace=True)`.

Armadilha! O OPF pode não convergir (`OPFNotConverged`) quando a carga é alta demais para os limites escolhidos. Não é um bug: é a física dizendo que essa hora exigiria corte de carga. Envolve o `rundcopp` em `try/except` e registre a hora como `NaN` (foi o que fizemos nos painéis das Sessões 1-2).

Armadilha! Coordenadas: 2.x vs. 3.x. No `pandapower` 2.x as coordenadas vivem em `net.bus_geodata`; no 3.x são a coluna GeoJSON `net.bus.geo`. Os notebooks do curso trazem o helper `coords_das_barras(net)` compatível com ambas.

Armadilha! Logging tagarela. Para silenciar os avisos repetitivos: `import logging; logging.getLogger("pandapower").setLevel(logging.CRITICAL)`.

Dica. Documentação oficial: <https://pandapower.readthedocs.io>. O tutorial “minimal example” de lá é uma ótima segunda leitura depois deste guia.

3 O ecossistema PySAL: econometria espacial em Python

Esta é, provavelmente, a parte mais nova do curso para vocês. Três pacotes irmãos, com papéis bem separados:

Pacote	Papel	O que usamos
<code>libpysal</code>	a matriz de pesos W	<code>weights.W</code> , <code>lat2W</code> , <code>transform="r"</code>
<code>esda</code>	estatística exploratória	<code>Moran</code> , <code>Moran_Local</code> (LISA)
<code>spreg</code>	os estimadores de regressão	<code>OLS</code> , <code>ML_Lag</code> (SAR/SDM), <code>ML_Error</code> (SEM)

3.1 Construir a matriz W

Para grades regulares (Lab 3) existe um atalho; para redes elétricas construímos W a partir do grafo e convertemos para o objeto da `libpysal`:

```
from libpysal.weights import lat2W, W as PysalW
import numpy as np

# (a) grade regular L x L, contiguidade rook
w = lat2W(8, 8, rook=True)
w.transform = "r" # padroniza por linhas: cada linha soma 1

# (b) de uma matriz densa qualquer (ex.: adjacencia da rede eletrica)
def a_pysal(Wm, ids):
    soma = Wm.sum(axis=1); soma = np.where(soma == 0, 1.0, soma)
    Wn = Wm / soma[:, None] # padronizacao por linhas
    viz = {ids[i]: [ids[j] for j in np.where(Wn[i] > 0)[0]]
```

```

        for i in range(len(ids)):
    pesos = {ids[i]: [Wn[i, j] for j in np.where(Wn[i] > 0)[0]]
        for i in range(len(ids))}
    return PysalW(viz, pesos, silence_warnings=True)

```

Dica. A padronização por linhas (`transform="r"`) faz com que Wy seja a *média dos vizinhos* — é o formato que todos os estimadores esperam. Esquecê-la é o erro número 1.

3.2 Autocorrelação: Moran global e local (LISA)

```

from esda.moran import Moran, Moran_Local

mi = Moran(y, w, permutations=999)
print(mi.I, mi.p_sim)           # estatística e p-valor por permutacao

lisa = Moran_Local(y, w, permutations=999, seed=2026)
sig = lisa.p_sim < 0.05
# lisa.q: 1=HH (alto entre altos), 2=LH, 3=LL, 4=HL

```

3.3 spreg: o contrato da biblioteca

Todos os estimadores compartilham a mesma interface — e os mesmos quatro pontos onde todo mundo tropeça na primeira vez:

Argumento	O que espera	Erro típico
<code>y</code>	array <i>coluna</i> (<code>n, 1</code>) → <code>y.reshape(-1, 1)</code>	passar (<code>n,</code>) plano
<code>X</code>	matriz (<code>n, k</code>) sem coluna de uns	duplicar a constante
<code>w</code>	objeto <code>W</code> padronizado por linhas	passar o <code>ndarray</code> cru
<code>name_x</code>	lista de rótulos	<i>summary</i> ilegível

Passo 1 — OLS com diagnóstico espacial (sempre o ponto de partida):

```

import spreg
ols = spreg.OLS(y.reshape(-1, 1), X, w=w, spat_diag=True,
               moran=True, name_x=["betweenness", "demanda", "
               exposicao"])
print(ols.summary)           # leia o bloco "DIAGNOSTICS FOR SPATIAL
                             DEPENDENCE"

```

Passo 2 — ler a bateria LM e decidir o modelo (regra de Anselin):

Padrão dos testes <i>robustos</i>	Veredito
Robust LM (lag) rejeita; Robust LM (error) não	SAR (<code>ML_Lag</code>): dependência via Wy
Robust LM (error) rejeita; Robust LM (lag) não	SEM (<code>ML_Error</code>): choque comum no erro
ambos rejeitam fortemente	considerar SDM (ver abaixo)
nenhum rejeita	fique no OLS

Passo 3 — estimar o modelo espacial:

```

sar = spreg.ML_Lag(y.reshape(-1, 1), X, w=w, method="full",
                  name_x=["betweenness", "demanda", "exposicao"])
rho = float(sar.rho)           # o parametro espacial

```

```

betas = np.ravel(sar.betas)          # [constante, b1, ..., bk, rho]
zs     = np.ravel(sar.z_stat)        # estatísticas z (mesma ordem)

sem = spreg.ML_Error(y.reshape(-1, 1), X, w=w,
                    name_x=["betweenness", "demanda", "exposicao"])
lam = float(sem.lam)

```

Armadilha! A ordem de betas. No ML_Lag, o vetor `sar.betas` vem como $[\beta_0, \beta_1, \dots, \beta_k, \rho]$ — a constante primeiro e o ρ no final. Com `slx_lags=1` (SDM), a ordem passa a ser $[\beta_0, \beta_1, \dots, \beta_k, \theta_1, \dots, \theta_k, \rho]$, onde os θ são os coeficientes de WX .

Passo 4 — SDM (Durbin espacial), quando o X do vizinho também importa:

```

sdm = spreg.ML_Lag(y.reshape(-1, 1), X, w=w, method="full",
                  slx_lags=1,          # acrescenta as colunas WX
                  name_x=["betweenness", "demanda", "exposicao"])

```

Passo 5 — efeitos direto / indireto / total (LeSage & Pace): num modelo espacial, β não é o efeito de x sobre y . A matriz de efeitos do regressor r é $S_r = \beta_r (\mathbb{I} - \rho W)^{-1}$ e dela saem as três grandezas que vão para o relatório:

```

n = len(y)
A = np.linalg.inv(np.eye(n) - rho * np.array(w.full()[0]))
Sr = betas[1] * A          # regressor 1 (apos a constante)

direto = Sr.diagonal().mean()      # efeito na propria barra
total  = Sr.sum(axis=1).mean()     # = beta_r / (1 - rho) no SAR
indireto = total - direto          # o spillover

```

Armadilha! Reportar β como “o efeito” é o erro conceitual mais comum em trabalhos com SAR. O efeito total é $\beta_r / (1 - \rho)$ — com $\rho = 0,5$, o β subestima o efeito pela metade. Foi o tema central da Parte V e do Lab 3.

Armadilha! OLS com Wy “na mão” é inconsistente. Incluir Wy como regressor num `lstsq`/OLS comum produz $\hat{\rho}$ viesado *mesmo com* $n \rightarrow \infty$ (vimos viés de +0,3 no Lab 3). Use sempre a máxima verossimilhança (ML_Lag/ML_Error), que paga o Jacobiano $\log|\mathbb{I} - \rho W|$.

Dica. Documentação: <https://pysal.org> (guarda-chuva), <https://pysal.org/spreg> (estimadores, com exemplos completos de summary).

4 networkx e scipy em meia página

```

import networkx as nx
G = nx.Graph(); G.add_nodes_from(barras); G.add_edge(i, j)

dict(G.degree())          # grau de cada no
nx.betweenness_centrality(G, normalized=False) # intermediacao
nx.is_connected(G); list(nx.connected_components(G))
A = nx.to_numpy_array(G, nodelist=sorted(G.nodes()), weight=None)

# espectro do Laplaciano (Sessão 1):
import numpy as np
L = np.diag(A.sum(axis=1)) - A
autovalores, autovetores = np.linalg.eigh(L)      # ordenados!
lam2, fiedler = autovalores[1], autovetores[:, 1]

```

```
from scipy.stats import spearmanr
spearmanr(x, y).statistic          # correlacao de
    ranking
```

Armadilha! Sempre passe `nodelist=sorted(G.nodes())` ao `to_numpy_array` — sem isso a ordem das linhas da matriz não é garantida e os índices deixam de corresponder às barras.

5 Mapa: comando → onde aparece no curso

Comando / conceito	Material
<code>pn.case_ieee30</code> , <code>rundcpp</code> , Laplaciano, λ_2 , Fiedler	Sessão 1 (§1–§3), Lab 1 (Bloco A)
PTDF, betweenness elétrica	Sessão 1 (§5), Lab 1 (Bloco B)
Cascatas de Motter–Lai	Sessão 1 (§6), Lab 1 (Blocos C–D)
<code>rundcpp</code> , <code>lam_p</code> , <code>res_cost</code> , limites <code>max_i_ka</code>	Sessão 2 (§1–§2), Lab 2 (Bloco A)
<code>create_poly_cost</code> , <code>create_gen</code> , <code>create_line_*</code>	Lab 2 (Blocos B–D), Lab 4 (Bloco C)
Moran, Moran_Local (LISA)	Sessão 2 (§4–§5), Lab 3 (Bloco E)
<code>spreg.OLS</code> + bateria LM	Sessão 2 (§5.3), Lab 3 (Bloco C)
<code>ML_Lag</code> , <code>ML_Error</code> , efeitos LeSage–Pace	Sessão 2 (§5.4–5.5), Lab 3 (Blocos B, D)
<code>slx_lags=1</code> (SDM)	Lab 3 (Bloco D)
<code>lat2W</code> , DGP sintético $(\mathbb{I} - \rho W)^{-1}$	Lab 3 (Bloco A), Lab 4 (Bloco A)
Colunas de $(\mathbb{I} - \rho W)^{-1}$, regressão inversa	Sessão 2 (§6), Lab 4

6 Solução de problemas frequentes

ModuleNotFoundError dentro do Jupyter. O kernel do notebook não é o do ambiente onde você instalou. Com o `venv` ativado, rode `pip install ipykernel && python -m ipykernel install -user -name lapsee` e selecione o kernel `lapsee` no Jupyter.

Avisos do numba / lightsim2grid. Inofensivos. O `pandapower` usa esses pacotes opcionalmente para acelerar; sem eles tudo funciona, só um pouco mais devagar.

error: externally-managed-environment (Linux). Você tentou `pip install` fora de um `venv`. Crie o `venv` (Opção A) ou adicione `-break-system-packages`.

OPFNotConverged. Não é defeito: a combinação carga $\times$ limites não tem solução factível. Trate com `try/except` (Seção 2, comando 7) ou afrouxe os limites.

Resultados diferentes dos notebooks. Confira o `RANDOM_SEED = 2026` e as versões da tabela da página 1. Pequenas diferenças numéricas (10^{-8}) entre sistemas operacionais são normais.

Gráficos não aparecem. Em scripts `.py` adicione `import matplotlib.pyplot as plt; plt.show()`. Em notebooks, nada a fazer.

Dúvidas de instalação? Tragam o erro completo (print da tela) para o primeiro laboratório — os 10 minutos iniciais são reservados para isso.